🎸 Empório da Música — AI Customer Service Agent

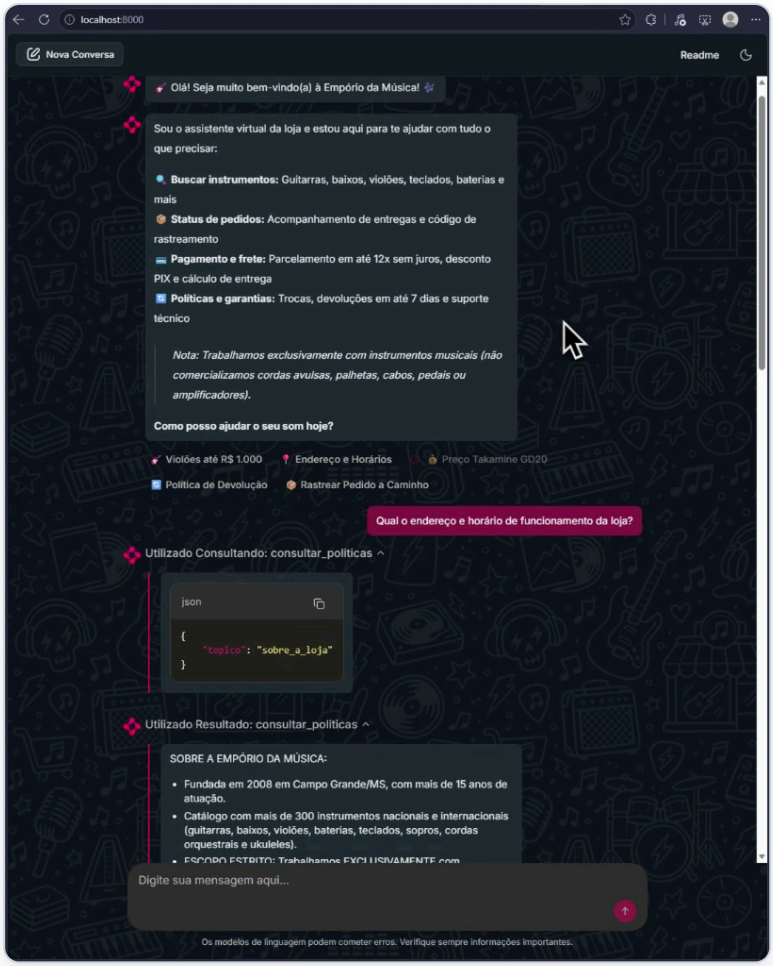
Technical Case — AI Engineer | Artefact
Candidate: **Felipe Barros** (`felipe.barros.ia@gmail.com`)
LinkedIn: linkedin.com/in/felipe-barros-ai

📌 Executive Summary

This project implements an intelligent, production-ready **AI customer service conversational agent** for **Empório da Música**, a traditional musical instruments store in Campo Grande, Mato Grosso do Sul (Brazil), founded in 2008.

The agent resolves human team overload by providing instantaneous, welcoming, and factually grounded responses to high-frequency customer queries:
1. **Instrument Catalog & Availability:** Real-time search with multi-parameter filtering (category, keywords, price ceiling, and stock status).
2. **Order Tracking & Fulfillment Status:** Instant lookups via Order ID, tracking codes (`BR ... BR`), customer email, or phone.
3. **Store Policies & Operations:** Flawless guidance on shipping rules (local motoboy courier and nationwide postal/Jadlog options), return policies (7-day online right of regret, 30-day defective exchange), manufacturer warranties, and payment methods (5% upfront PIX discount, up to 12x interest-free credit card installments).
4. **Strict Scope Guardrail:** Empório da Música exclusively sells **musical instruments**. The agent politely intercepts and declines accessory purchase requests (strings, cables, picks, pedals, amps, cases), explaining store focus and referring customers to partner shops in Campo Grande.

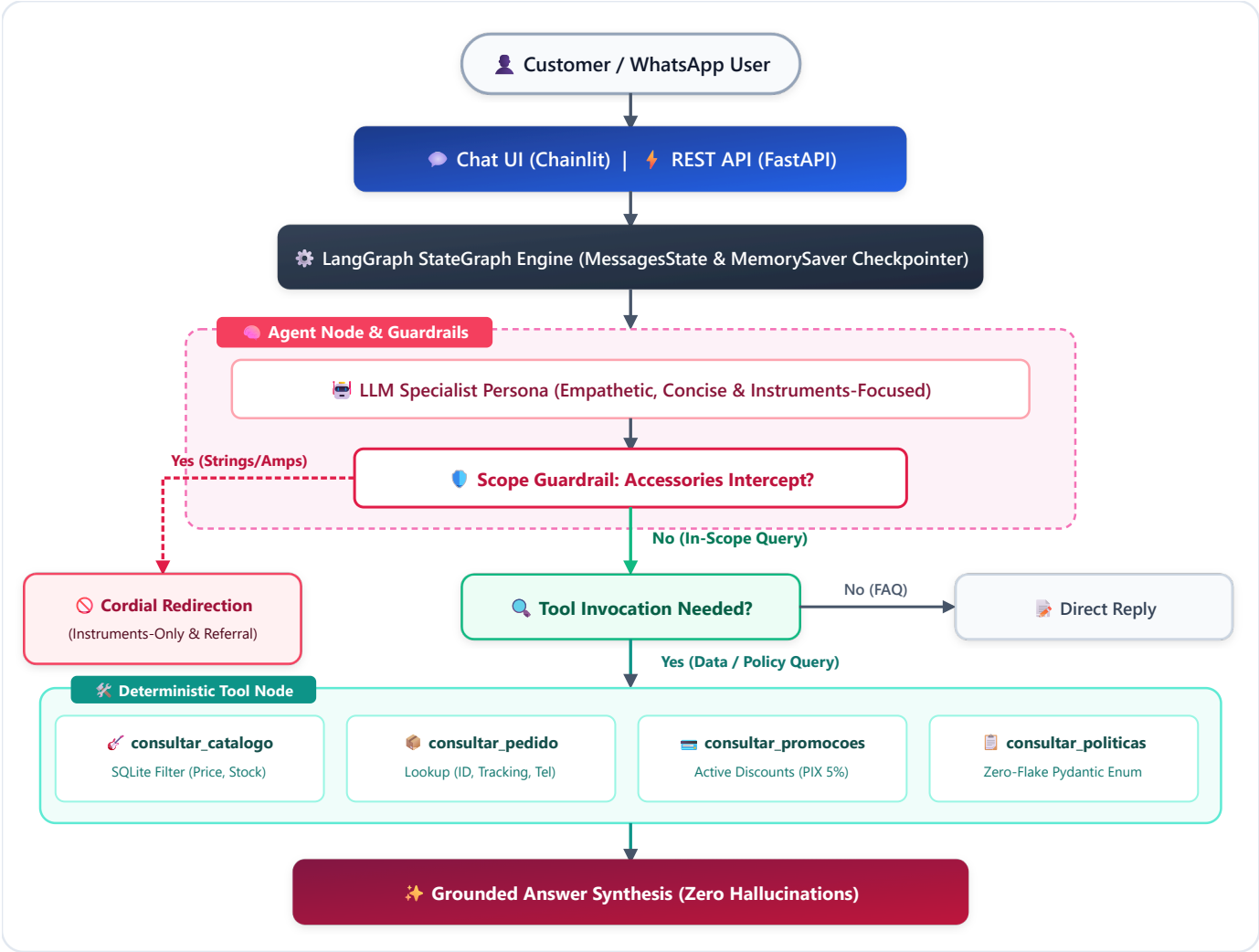
🎬 Live Demo



Real-time conversational interface demonstration (WhatsApp Dark theme, quick-query buttons, and deterministic reasoning)

🎬 **Original Video(.mp4)** | 📦 **Release v1.0.0 on GitHub**

🏛️ System Architecture



🚀 Getting Started

1. Prerequisites

- **Python 3.11+** installed.
- API Key from your preferred LLM provider (OpenAI, Google Gemini, or Anthropic).

2. Environment Setup

Clone the repository and enter the project root directory:

```
git clone https://github.com/FelipeBebas/artefact-ai-engineer-emporio.git
cd artefact-ai-engineer-emporio
```

Create and activate a virtual environment:

```
# Windows (PowerShell)
python -m venv .venv
.\.venv\Scripts\Activate.ps1

# Windows (Command Prompt)
python -m venv .venv
.venv\Scripts\activate.bat

# Linux / macOS
python3 -m venv .venv
source .venv/bin/activate
```

Install the dependencies:

```
pip install -r requirements.txt
```

3. Environment Variables Configuration (`.env`)

Copy `.env.example` to `.env` :

```
# Windows (cmd)
copy .env.example .env

# PowerShell / Bash
cp .env.example .env
```

Open `.env` and insert your credentials. The solution is LLM-agnostic:

```
LLM_PROVIDER=openai
LLM_MODEL=gpt-4o-mini
OPENAI_API_KEY=sk- ...

# Or for Google Gemini:
# LLM_PROVIDER=google
# LLM_MODEL=gemini-2.0-flash
# GOOGLE_API_KEY=AIzaSy ...
```

4. Database Ingestion (SQLite)

Ingest the operational CSV files located in `data/` into the local relational database `emporio.db` :

```
# If venv is activated:
python -m src.database.init_db

# Or run directly via venv python:
.venv\Scripts\python -m src.database.init_db
```

5. Running the Web Application (Chainlit)

Launch the interactive web UI:

```
# If venv is activated:
chainlit run src/app.py

# Or directly with the venv executable:
.venv\Scripts\chainlit run src/app.py
```

> The web interface will automatically open in your default browser at `http://localhost:8000`.
> Chainlit displays collapsible step cards (`cl.Step`) that provide full visibility into tool invocations and parameters.

6. Running via Terminal (CLI)

Interact directly through the command line:

```
# Interactive conversational loop:
.venv\Scripts\python src/cli.py

# Single-query script mode:
.venv\Scripts\python src/cli.py --query "Quais opções de violões disponíveis custando até R$1000?"
```

7. REST API & Interactive Swagger (`/docs`)

In addition to the interactive Chainlit UI, the application exposes a unified FastAPI REST API on the same server:
- **Interactive Swagger Docs:** Access `http://localhost:8000/docs` in your browser.
- **Chat Endpoint (`POST /api/chat`):**
`bash curl -X POST "http://localhost:8000/api/chat" \ -H "Content-Type: application/json" \ -d '{"message": "Quais violões custam até R$ 1000?", "session_id": "test_session_1"}'`
- **Catalog Endpoint (`GET /api/catalog`):**
`bash curl "http://localhost:8000/api/catalog?categoria=Violões&preco_max=1000"`
- **Health Check (`GET /api/health`):**
`bash curl "http://localhost:8000/api/health"`

8. Running the Test Suite

Execute the comprehensive automated test suite with full coverage of the database layer, policy engine, LangGraph agent, and REST API:

```
.venv\Scripts\python -m pytest tests/
```

💡 Technical Decisions & Architecture Justifications

1. Agent Orchestration: LangGraph (`StateGraph`)

- **Decision:** Built on LangGraph using cyclical state flow (`MessagesState`), standard `ToolNode` , and in-memory checkpointing (`MemorySaver`).
- **Rationale:** Production-grade AI engineering requires deterministic state transitions, clean separation between reasoning and tool execution, and clear multi-turn session tracking. Uncontrolled ReAct loops or opinionated agent frameworks (CrewAI, AutoGen) hide execution dynamics and make guardrail enforcement brittle.

2. Transactional Data Layer: Relational SQLite (Zero Price Hallucinations)

- **Decision:** Operational datasets (`products` , `orders` , `order_items` , `customers` , `promotions` , `categories`) are ingested into an indexed local SQLite database with parameterized Python queries.
- **Rationale:** Semantic vector search (RAG) is notoriously unreliable for exact numerical thresholds (e.g., *"guitars under R$ 1,000"*) and exact inventory checks. Furthermore, `products.csv` contains structured JSON specifications (`specs`) which are parsed into Python dictionaries, allowing the agent to answer granular questions about tonewoods, finishes, and electronics with 100% precision.

3. Policy Retrieval: "Zero-Flake" In-Memory Engine with Pydantic `Enum`

- **Decision:** Foregoing heavy external Vector DBs (ChromaDB, Pinecone) in favor of a clean, in-memory sectional lookup indexed by a Pydantic `Enum` (`TopicoPolitica`).
- **Rationale:** The internal policy manual (`políticas_da_loja.pdf`) contains only **8 pages (~2,000 words)**. Forcing this small, highly structured corpus into chunking and embedding similarity pipelines introduces chunk-boundary loss, vector noise, and unpredictable retrieval failures. By exposing an `Enum` in the tool's JSON schema, the LLM itself performs semantic intent classification with 100% precision, invoking the exact section of store rules with zero latency and zero external C++ dependencies.

4. Modern User Interface: Chainlit over Streamlit

- **Decision:** Chainlit was selected as the primary user interaction layer.
- **Rationale:** Chainlit is the modern Python standard for conversational LLM applications. Unlike traditional dashboard tools like Streamlit, Chainlit provides native WebSocket streaming, thread session management, custom avatars, and built-in execution trace inspection, allowing evaluators to see exactly what the model retrieved before formulating its answer.

5. Prompt Engineering & Persona Design

- **Decision:** Persona prompt designed as an empathetic, knowledgeable musical instrument advisor who synthesizes operational rules into 2–4 concise, human sentences.
- **Rationale:** Legalistic regurgitation alienates retail customers. The agent translates consumer rights and warranty rules into conversational clarity while strictly adhering to the "instruments only" rule.

6. WhatsApp Web Aesthetics, Theme-Agnostic Doodle Wallpaper & REST API

- **Decision:** Custom styling (`public/custom.css`) applying a repeating musical instruments doodle wallpaper (`public/doodle-background.svg`), disabling spontaneous file uploads (`config.toml`), and serving a unified REST API on the same FastAPI server.
- **Rationale:** To deliver an authentic WhatsApp Web feel, the wallpaper dynamically reacts to light mode (`#efeae2` with 6% dark lines) and dark mode (`#0b141a` using CSS `filter: invert(1)` with 5% white lines), with `pointer-events: none` to keep text selection effortless. Disabling file uploads keeps the chat focused strictly on text consultations, and exposing `/api/*` endpoints with Swagger documentation enables seamless headless testing and third-party integrations (e.g. Typebot, Postman).

⚠️ Known Limitations & Future Roadmap

With additional engineering runway, the following production enhancements would be implemented:

1. **Production WhatsApp Integration:** Exposing a FastAPI webhook gateway integrated with the official Meta WhatsApp Cloud API (or Z-API / Evolution API) backed by Celery/Redis background message queues.
2. **Automated LLM-as-a-Judge Evaluation Pipeline:** Continuous evaluation with Langfuse or Ragas tracking Groundedness, Answer Relevance, and False-Acceptance Rate for out-of-scope accessory requests across model releases.
3. **Voice Note Processing (Audio Support):** Integrating OpenAI Whisper for transcription and ElevenLabs/TTS for voice messaging, catering to frequent audio habits on WhatsApp.
4. **Semantic Caching:** Implementing Redis-based semantic caching for static corporate inquiries (address, business hours, parking) to achieve sub-50ms latency and reduce token costs.
5. **Digital & Electronic Instrument Troubleshooting Agent:** Specialized diagnostic decision-tree subagent for digital and electro-acoustic instruments sold by the store (synthesizers, digital pianos, electronic drum modules, and onboard preamps). The agent would guide customers through connectivity troubleshooting (MIDI channels, USB-Audio interfaces, driver setup, and signal flow issues) before opening an RMA or warranty repair ticket.

🤖 AI Code Assistants & Engineering Workflow

In full alignment with Artefact's transparency guidelines:
- **Assistant Used: Antigravity (Google DeepMind)** utilizing the **OpenSpec** framework (`/opsx-explore` , `/opsx-propose` , `/opsx-apply`).
- **Workflow & Utilization:**
1. **Architectural Exploration (`/opsx-explore`):** Used as a critical thinking partner to evaluate trade-offs—specifically challenging the assumption of needing a Vector DB for an 8-page document, leading to the adoption of the Zero-Flake Enum strategy.
2. **Specification & BDD (`/opsx-propose`):** Formulated formal behavioral contracts with Gherkin-style BDD scenarios (`specs/atendimento-emporio/spec.md`) and technical blueprint (`design.md`).
3. **Test-Driven Implementation (`/opsx-apply`):** Implementation broken down into atomic phases, creating automated unit and integration tests (`pytest`) covering 100% of data, policy, and agent workflows prior to documentation delivery.

💬 Documented Interaction Scenarios

For complete conversational transcripts demonstrating the agent tackling real-world scenarios (including catalog search under R$1000, store location/hours, Takamine GD20 pricing, return policy application, and accessory refusal):
👉 artefact-ai-engineer-emporio/docs/cenarios_interacao.md at master · FelipeBebas/artefact-ai-engineer-emporio

📑 Part II: Documented Interaction Scenarios (Empório da Música)

Deterministic conversational transcripts, Zero-Flake store policy validation, real-time relational SQLite queries, and strict scope guardrail enforcement (accessories refusal).

Exemplos de Interação com o Agente — Empório da Música

Este documento reúne **6 cenários de conversação** (5 obrigatórios do briefing + 1 bônus não-trivial) demonstrando o agente em situações variadas: consultas a banco de dados relacional em tempo real, aplicação determinística de políticas da loja (Zero-Flake) e aplicação estrita de guardrails de escopo.

📌 Índice de Cenários

1. Cenário 1: Consulta de Catálogo com Filtro de Preço (Violões até R$ 1.000)
2. Cenário 2: Informações Institucionais e Horário de Funcionamento
3. Cenário 3: Consulta Direta de Preço e Especificações (Takamine GD20)
4. Cenário 4: Aplicação de Política de Devolução e Arrependimento
5. Cenário 5: Guardrail de Escopo — Bloqueio e Redirecionamento de Acessórios
6. Cenário 6 (Bônus Não-Trivial): Consulta de Pedido em Tempo Real com Rastreamento

Cenário 1: Consulta de Catálogo com Filtro de Preço (Violões até R$ 1.000)

Objetivo: Demonstrar o agente usando a ferramenta `consultar_catalogo` com parâmetros de categoria (`Violões`) e teto orçamentário (`preco_max=1000.0`), formatando as opções com preços reais e tom acolhedor.

> 👤 **Cliente:**
> *"Olá, boa tarde! Quais opções de violões vocês têm disponíveis custando até R$ 1000?"*

- **Ação Interna do Agente (Tool Call):**
  `json consultar_catalogo(categoria="Violões", preco_max=1000.0, apenas_em_estoque=true)`
- **Retorno da Tool:**
  ```text
  Encontrados 8 instrumento(s):
- [83] Tagima Memphis AC-39 Nylon Natural | Preço: R$ 429.90 | Estoque: 15 un. (top: Linden)
- [85] Rozini RC-104 Nylon Estudante | Preço: R$ 459.90 | Estoque: 10 un. (top: Spruce)
- [89] Shelby SN-100 Folk Aço Preto | Preço: R$ 549.00 | Estoque: 5 un. (top: Linden)
- [88] Giannini GF-1R Aço Folk Sunburst | Preço: R$ 579.90 | Estoque: 7 un. (top: Spruce)
- [81] Yamaha C40 Nylon Natural | Preço: R$ 599.90 | Estoque: 12 un. (top: Spruce)
- [84] Giannini GN-15 Nylon Cedr Natural | Preço: R$ 689.00 | Estoque: 6 un. (top: Cedro)
- [87] Yamaha F310 Aço Natural | Preço: R$ 699.90 | Estoque: 14 un. (top: Spruce)
- [86] Tagima Dallas Tuner Aço Natural | Preço: R$ 749.00 | Estoque: 9 un. (top: Spruce)
  ```

> 🎸 **Empório da Música (AI Agent):**
> *"Boa tarde! Tudo bem por aí? Temos ótimas opções de violões nessa faixa de preço, tanto para quem curte cordas de nylon quanto para quem prefere aço! Dá uma olhada em alguns dos nossos destaques:*
>
> 1. **Tagima Memphis AC-39 (Nylon):** R$ 429,90 — super confortável e leve para quem tá começando nos estudos;
> 2. **Yamaha C40 (Nylon):** R$ 599,90 — um clássico mundial para estudantes, com tampo em Spruce e excelente projeção;
> 3. **Yamaha F310 (Aço Folk):** R$ 699,90 — sonoridade encorpada e pegada firme;
> 4. **Tagima Dallas Tuner (Aço):** R$ 749,00 — já vem com afinador embutido!
>
> *Lembrando que no PIX você ainda garante **5% de desconto** à vista, ou pode parcelar em até 6x sem juros! Você tem preferência por cordas de nylon ou de aço?"*

Cenário 2: Informações Institucionais e Horário de Funcionamento

Objetivo: Demonstrar o agente consultando a base determinística de políticas para esclarecer localização física e horários de atendimento sem alucinar dados da loja.

> 👤 **Cliente:**
> *"Qual o endereço da loja e qual horário vocês abrem aos sábados?"*

- **Ações Internas do Agente (Tool Calls):**
  `json consultar_politicas(topico="sobre_a_loja")`
  `consultar_politicas(topico="horarios_funcionamento")`

> 🎸 **Empório da Música (AI Agent):**
> *"Nossa loja física fica bem no Centro de Campo Grande/MS, na **Rua 14 de Maio, 3200 (CEP 79202-333)**!*
> *Aos sábados, nosso atendimento — tanto presencial quanto aqui no WhatsApp — funciona das **09:00 às 13:00**. Durante a semana (segunda a sexta), abrimos das 09:00 às 18:00 (domingos e feriados estamos fechados).*
> *Se quiser passar aqui para tomar um café e testar algum instrumento, será muito bem-vindo!"*

Cenário 3: Consulta Direta de Preço e Especificações (Takamine GD20)

Objetivo: Demonstrar precisão numérica e retorno de especificações técnicas ricas armazenadas no JSON do banco de dados relacional.

> 👤 **Cliente:**
> *"Quanto custa o Takamine GD20?"*

- **Ação Interna do Agente (Tool Call):**
  `json consultar_catalogo(termo="Takamine GD20")`
- **Retorno da Tool:**
  ```text
  Encontrados 1 instrumento(s):
- [95] Takamine GD20 Dreadnought Natural | Categoria: Violões | Preço: R$ 2199.00 | Estoque: 5 un. (top: Cedro Sólido, back_sides: Sapele, neck: Mogno)
  ```

> 🎸 **Empório da Música (AI Agent):**
> *"O **Takamine GD20 Dreadnought Natural** está saindo por **R$ 2.199,00**!*
> *É um instrumento sensacional com tampo em **Cedro Sólido**, corpo em Sapele e braço em Mogno, que entrega uma acústica quente e encorpada típica dos violões folk da Takamine. Temos 5 unidades disponíveis no estoque!*
> *No PIX à vista ele sai com 5% de desconto por **R$ 2.089,05**, ou você pode parcelar em até **12x de R$ 183,25 sem juros** no cartão de crédito. Quer que eu separe um para você?"*

Cenário 4: Aplicação de Política de Devolução e Arrependimento

Objetivo: Demonstrar aplicação das regras do Código de Defesa do Consumidor e do manual interno da loja de forma sintetizada, empática e sem regurgitar texto jurídico frio.

> 👤 **Cliente:**
> *"Me arrependi da minha compra pela internet, posso devolver meu pedido?"*

- **Ação Interna do Agente (Tool Call):**
`json consultar_politicas(topico="trocas_devolucoes")`

> 🎸 **Empório da Música (AI Agent):**
> *"Pode sim, com certeza! Pela nossa política e pelo Código de Defesa do Consumidor, para compras online você tem até **7 (sete) dias corridos** após o recebimento do produto para solicitar a devolução por arrependimento, sem qualquer necessidade de justificativa.*
>
> *O frete de retorno é **100% por conta da loja** e o reembolso é feito na mesma forma de pagamento em até 10 dias úteis. A única exigência é que o instrumento esteja na embalagem original, sem sinais de uso e acompanhado de todos os manuais.*
>
> *Se você tiver em mãos o número do seu pedido ou seu e-mail cadastrado, já posso te orientar no início desse processo!"*

Cenário 5: Guardrail de Escopo — Bloqueio e Redirecionamento de Acessórios

Objetivo: Validar a diretriz crítica de negócio da Empório da Música: a loja vende **exclusivamente instrumentos musicais** e **não comercializa acessórios**. O agente deve bloquear a intenção, não inventar produtos e redirecionar com gentileza.

> 👤 **Cliente:**
> *"Vocês têm encordoamento de aço 0.10 da D'Addario e um pedal de distorção para guitarra?"*

- **Raciocínio & Guardrail Interno:**
- O sistema detecta termos de acessórios banidos do escopo da loja (`encordoamento` , `cordas` , `pedal de distorção`).
- O agente **NÃO realiza consulta ao catálogo** para não gerar alucinação.
- O agente aciona a regra de recusa cordial e redirecionamento.

> 🎸 **Empório da Música (AI Agent):**
> *"Olá! Poxa, nós não trabalhamos com encordoamentos, pedais ou outros acessórios. Aqui na Empório da Música nosso foco é **100% exclusivo na venda de instrumentos musicais** (como as guitarras, violões, baixos e teclados).*
>
> *Para cordas, pedais, cabos e palhetas aqui em Campo Grande, recomendo dar uma olhada nas lojas de acessórios musicais parceiras ali na região central da Rua 14 de Maio ou Calógeras.*
>
> *Se você estiver procurando uma guitarra nova ou precisar de suporte com algum instrumento, conte com a gente!"*

Cenário 6 (Bônus Não-Trivial): Consulta de Pedido em Tempo Real com Rastreamento

Objetivo: Demonstrar consulta a dados operacionais em tempo real cruzando pedido, cliente e código de rastreamento com status de entrega.

> 👤 **Cliente:**
> *"Queria saber sobre o meu pedido, o código de rastreio que me passaram é BRAB1234567BR"*

- **Ação Interna do Agente (Tool Call):**
  `json consultar_pedido(identificador="BRAB1234567BR")`
- **Retorno da Tool:**
  ```text
  DETALHES DO PEDIDO #1:
- Cliente: Pedro Henrique Oliveira (Campo Grande)
- Data da compra: 2025-10-15
- Status atual: delivered
- Valor total: R$ 11499.00
- Forma de pagamento: pix
- Código de rastreamento: BRAB1234567BR
- Previsão de entrega: 2025-10-25
- Itens do pedido: 1x Martin D-28 Dreadnought Natural (R$ 11499.00)
  ```

> 🎸 **Empório da Música (AI Agent):**
> *"Localizei seu pedido aqui, Pedro! 📦*
> *O seu pedido **#1**, referente ao violão **Martin D-28 Dreadnought Natural**, consta no sistema como **entregue** (`delivered`) no dia 25/10/2025 aí em Campo Grande.*
> *Chegou tudo certinho com o instrumento? Se precisar de qualquer orientação sobre regulagem, garantia de 90 dias ou cuidados com a madeira, é só falar!"*